

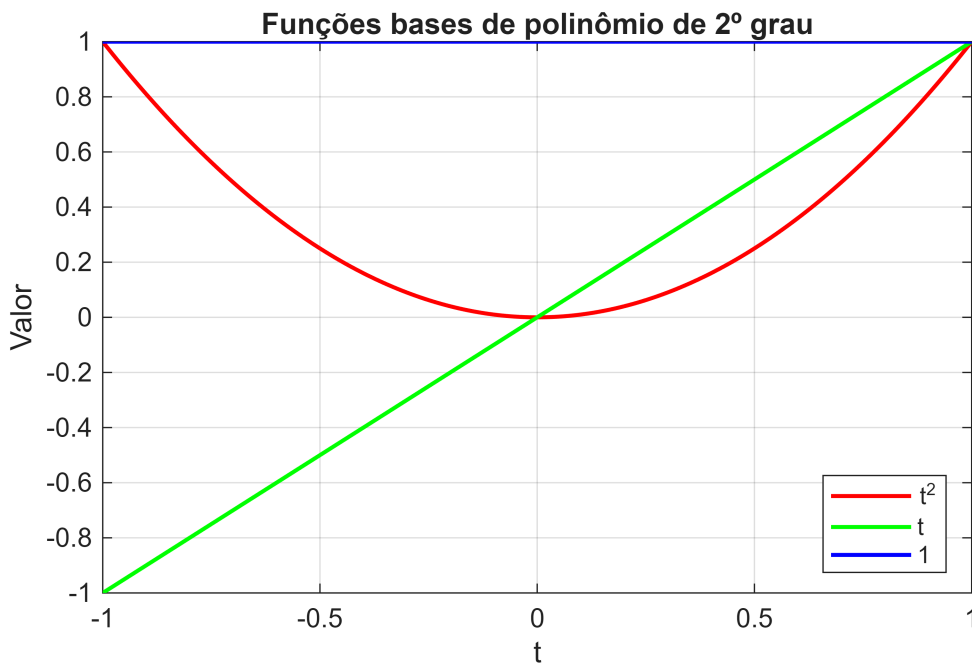
```
% Exercícios Computacionais - Semana 2
% Aluno: João Vitor Miranda Sousa, Nº USP: 14802702
```

```
%% Questão 1 - Polinômio de segundo grau
% Polinômio:  $P_2(t) = a \cdot t^2 + b \cdot t + c$ ,  $t \in [-1, 1]$ 
```

```
% Intervalo do tempo
t = -1:0.01:1;
```

```
% (a) Funções base:  $t^2$ ,  $t$  e 1
base1 = t.^2;
base2 = t;
base3 = ones(size(t));
```

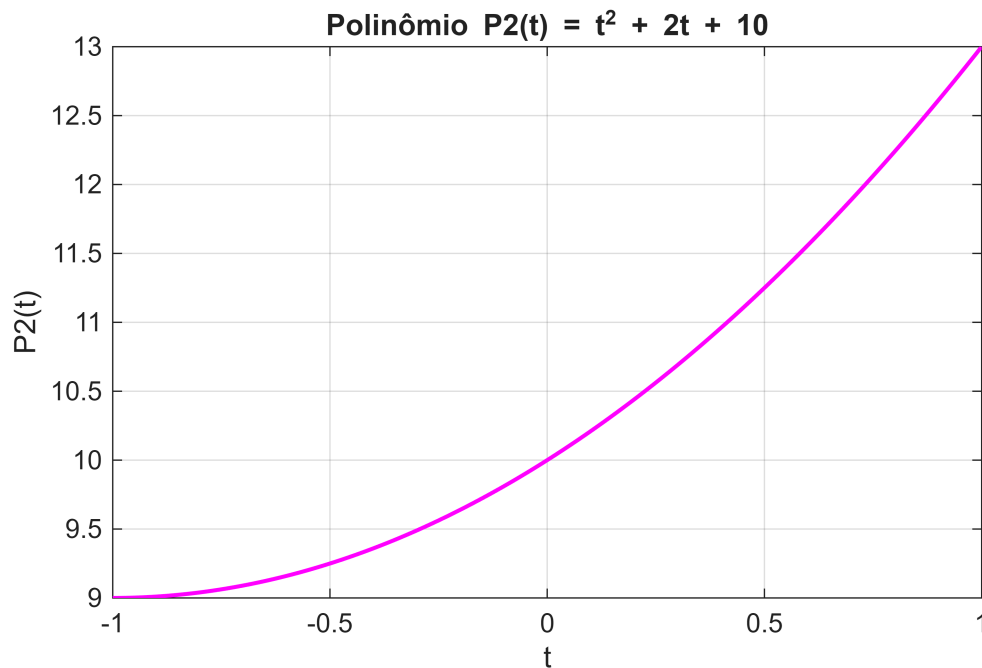
```
figure;
plot(t, base1, 'r', 'LineWidth', 1.5); hold on;
plot(t, base2, 'g', 'LineWidth', 1.5);
plot(t, base3, 'b', 'LineWidth', 1.5);
grid on; box on;
title('Funções bases de polinômio de 2º grau');
xlabel('t'); ylabel('Valor');
legend('t^2', 't', '1', 'Location', 'best');
```



```
% (b) Polinômio com a=1, b=2, c=10
a = 1; b = 2; c = 10;
P2 = a*t.^2 + b*t + c;
```

```
figure;
plot(t, P2, 'm', 'LineWidth', 1.5);
grid on; box on;
```

```
title('Polinômio P2(t) = t^2 + 2t + 10');
xlabel('t'); ylabel('P2(t)');
```



```
%% Questão 2 - Produto escalar
% g(t) = 5t^2 + 2t + 9
% h(t) = sin(2t) * log(5 + 3t)
% t ∈ [-1, 1]

% Definição das funções (funções anônimas no MATLAB)
g = @(t) 5*t.^2 + 2*t + 9;
h = @(t) sin(2*t).*log(5 + 3*t);

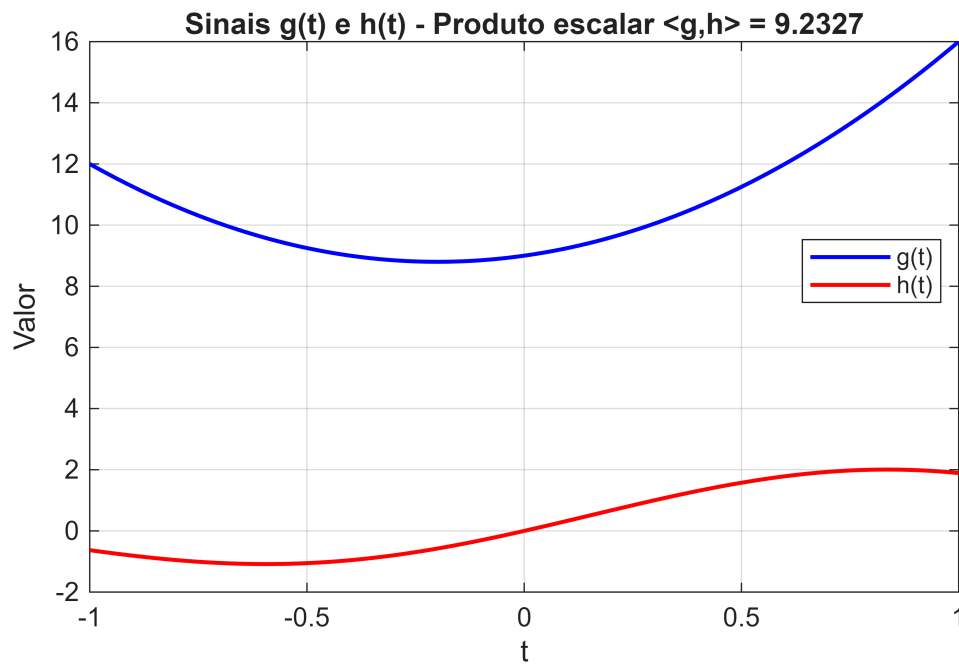
% Produto escalar via integração numérica
produto_escalar = integral(@(t) g(t).*h(t), -1, 1);

% Mostrar resultado no console (com 4 casas)
disp(sprintf('Produto escalar <g,h> = %.4f', produto_escalar));
```

Produto escalar <g,h> = 9.2327

```
% --- Gráfico adicional para ilustrar os sinais g(t) e h(t) ---
t_vals = linspace(-1,1,1000);
figure;
plot(t_vals, g(t_vals), 'b', 'LineWidth', 1.5); hold on;
plot(t_vals, h(t_vals), 'r', 'LineWidth', 1.5);
grid on; box on;
title(sprintf('Sinais g(t) e h(t) - Produto escalar <g,h> = %.4f',
produto_escalar));
xlabel('t'); ylabel('Valor');
```

```
legend('g(t)', 'h(t)', 'Location', 'best');
```



```
ax = gca; ax.YAxis.Exponent = 0; % evita notação científica sobrepondo
% -----

%% Questão 3 - Aproximação por Série de Taylor
% Sinal:  $x(t) = \cos(10\pi t)$ 
% Expansão de Taylor em torno de  $t=0$  usando as 5 primeiras bases.

% Usar variável simbólica diferente de 't' para evitar conflito
syms tau
x = cos(10*pi*tau);

% 'Order',10 → inclui termos até  $t^8$  (5 primeiros termos não nulos da série)
x_taylor5 = taylor(x, tau, 'ExpansionPoint', 0, 'Order', 10);

% Exibir no console a expressão simbólica encontrada
disp('Série de Taylor (5 primeiros termos):');
```

Série de Taylor (5 primeiros termos):

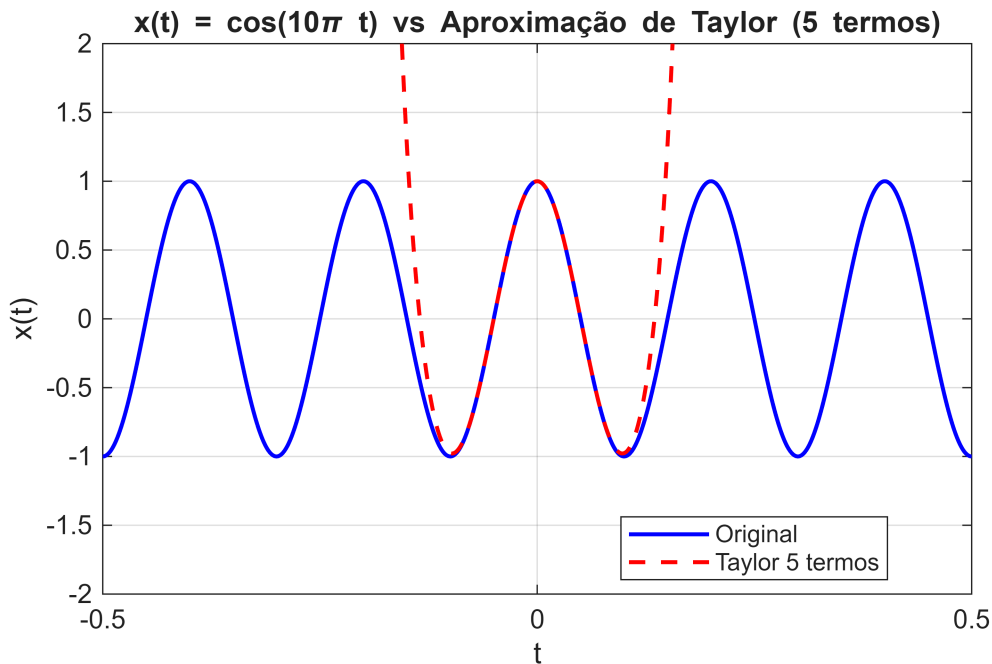
```
pretty(simplify(x_taylor5))
```

```

      8      8      6      6      4      4      2      2
156250 pi tau - 12500 pi tau + 1250 pi tau - 50 pi tau + 1
-----
      63          9          3
```

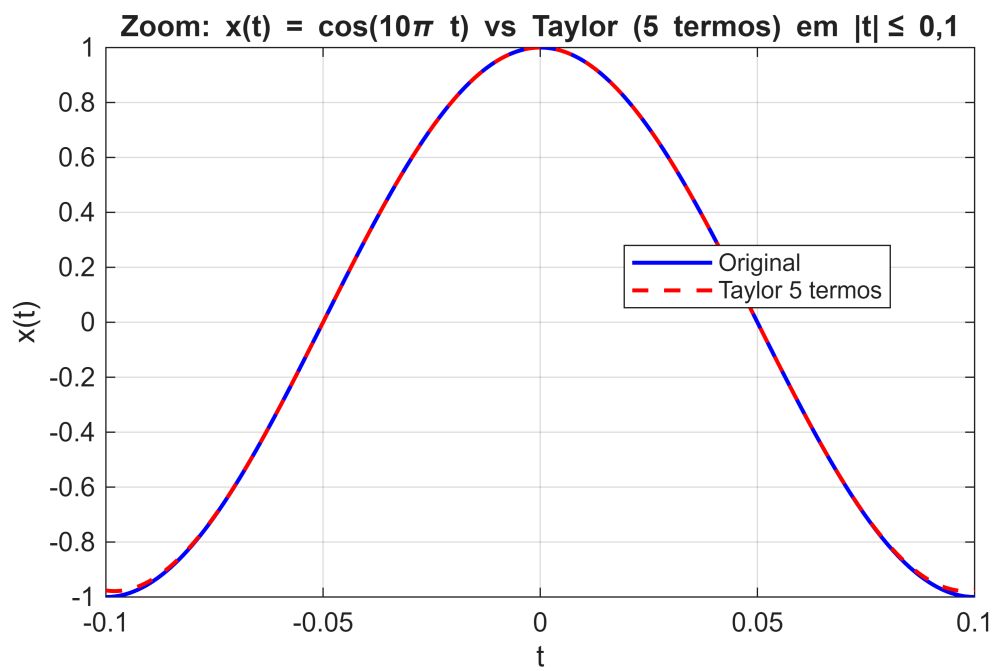
```
% --- Figura 1: intervalo amplo, mas com eixo Y limitado para visualização ---
t_vals = linspace(-0.5, 0.5, 2000);
x_original = cos(10*pi*t_vals);
x_aprox = double(subs(x_taylor5, tau, t_vals));
```

```
figure;
plot(t_vals, x_original, 'b', 'LineWidth', 1.5); hold on;
plot(t_vals, x_aprox, 'r--', 'LineWidth', 1.5);
grid on; box on;
title('x(t) = cos(10\pi t) vs Aproximação de Taylor (5 termos)');
xlabel('t'); ylabel('x(t)');
legend('Original', 'Taylor 5 termos', 'Location', 'best');
ax = gca; ax.YAxis.Exponent = 0;
ylim([-2 2]); % limita y para mostrar que a aproximação se perde longe de 0
```



```
% --- Figura 2: zoom perto de t=0 onde a aproximação é boa ---
t_zoom = linspace(-0.1, 0.1, 2000);
x_orig_zoom = cos(10*pi*t_zoom);
x_aprx_zoom = double(subs(x_taylor5, tau, t_zoom));

figure;
plot(t_zoom, x_orig_zoom, 'b', 'LineWidth', 1.5); hold on;
plot(t_zoom, x_aprx_zoom, 'r--', 'LineWidth', 1.5);
grid on; box on;
title('Zoom: x(t) = cos(10\pi t) vs Taylor (5 termos) em |t|\leq 0,1');
xlabel('t'); ylabel('x(t)');
legend('Original', 'Taylor 5 termos', 'Location', 'best');
```



```
ax = gca; ax.YAxis.Exponent = 0;
```